✦ Gemini

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# ------------------------------------------
# Step 1: Upload an image
# ------------------------------------------
print("Please upload an image file.")
uploaded = files.upload()

# Get uploaded file name
if uploaded:
    image_path = list(uploaded.keys())[0]
    print(f"Uploaded image: {image_path}")
else:
    print("No image uploaded. Please upload an image to proceed.")
    # You might want to add an exit() or raise an error here if an image is ma
    # For now, we'll just skip the processing if no image is uploaded.
    image_path = None

if image_path:
    # Read image in grayscale
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    # Check whether image is loaded
    if image is None:
        print("Error loading image. Please ensure it's a valid image file.")
    else:
        # ------------------------------------------
        # Step 2: Simulate Sensor Noise (Gaussian Noise)
        # ------------------------------------------
        mean = 0
        std = 25 # Standard deviation for Gaussian noise

        # Generate Gaussian noise of the same shape as the image
        noise = np.random.normal(mean, std, image.shape)

        # Add noise to the image
        noisy_image = image + noise

        # Limit pixel values between 0 and 255 and convert to uint8
        noisy_image = np.clip(noisy_image, 0, 255).astype(np.uint8)

        # ------------------------------------------
        # Step 3: Noise Reduction
        # ------------------------------------------
```

```
# Gaussian Filter: Blurs the image to remove noise but also softens ed
gaussian_filtered = cv2.GaussianBlur(noisy_image, (5,5), 0)

# Median Filter: Effective at removing salt-and-pepper noise and other
median_filtered = cv2.medianBlur(noisy_image, 5)

# Bilateral Filter (Edge Preserving): Smooths flat regions while prese
# Parameters: image, diameter of pixel neighborhood, sigmaColor, sigma
bilateral_filtered = cv2.bilateralFilter(noisy_image, 9, 75, 75)

# -----------------------------------------
# Step 4: Display Results
# -----------------------------------------
plt.figure(figsize=(15,10))

plt.subplot(2,3,1)
plt.imshow(image, cmap='gray')
plt.title("Original Image")
plt.axis('off')

plt.subplot(2,3,2)
plt.imshow(noisy_image, cmap='gray')
plt.title("Image with Sensor Noise")
plt.axis('off')

plt.subplot(2,3,3)
plt.imshow(gaussian_filtered, cmap='gray')
plt.title("Gaussian Filter")
plt.axis('off')

plt.subplot(2,3,4)
plt.imshow(median_filtered, cmap='gray')
plt.title("Median Filter")
plt.axis('off')

plt.subplot(2,3,5)
plt.imshow(bilateral_filtered, cmap='gray')
plt.title("Bilateral Filter (Edge Preserving)")
plt.axis('off')

plt.tight_layout()
plt.show()
```

```
Please upload an image file.
Browse... gPYCkddWs89JZOnVm7LRrB-fRyCR01QsQ5C7L6FjzZjlpGRFU8Mw_rCT4ZAyk8xO90KasSzj
gPYCkddWs89JZOnVm7LRrB-
fRyCR01QsQ5C7L6FjzZjlpGRFU8Mw_rCT4ZAyk8xO90KasSzjNJ60Me1JU2R9ZSoF-
eGff9O4TtSalHonGGVWr1A2dLZPDaDhNhw74mAa4eaC80yLynFJ53sxuezt8zTvYJMjGtye
jpeg) - 39400 bytes, last modified: n/a - 100% done
Saving gPYCkddWs89JZOnVm7LRrB-fRyCR01QsQ5C7L6FjzZjlpGRFU8Mw_rCT4ZAyk8xO90KasS
```

```
Uploaded image: gPYCkddWs89JZOnVm7LRrB-fRyCR01QsQ5C7L6FjzZjlpGRFU8Mw_rCT4ZAyk
```